BuildScan 3D: Mobile-to-Cloud 3D Gaussian Splatting for Metric Indoor Reconstruction

Project Team

Muhammad Umair 23I-0662
Zaid Fareed 23I-0829
Muhammad Tayyab 23I-0636

Session 2023-2027

Supervised by

Mr. Furqan Nasir



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Contents

Students' Submission i

Anti-Plagiarism Statement ii

Authors' Declaration iii

Abstract iv

Acknowledgments v

Executive Summary vi

List of Abbreviations viii

1 Introduction and Project Vision 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Problem Context and Motivation . . . 2
1.4 Project Vision . . . 2
1.5 Project Objectives . . . 2
1.6 Project Scope . . . 3
1.7 Project Modules . . . 3
1.8 Stakeholders and User Classes . . . 4
1.9 Success Criteria . . . 4
1.10 Constraints and Assumptions . . . 4
1.11 Ethical, Legal and DEI Considerations . . . 5

2 Research and Existing Solutions 6

2.1 Research Method / Approach . . . 6
2.2 Existing Solutions / Products . . . 6
2.3 Comparison of Existing Solutions . . . 7
2.4 Limitations and Gaps . . . 7
2.5 Proposed Solution . . . 8
2.6 Complex Computing Problem . . . 8

2.7 Technical Challenges . . . 9
2.8 Research Findings and Design Decisions . . . 10

3 Proposal Feedback and Iteration Plan **11**
3.1 Proposal Feedback Summary . . . 11
3.2 Proposal Action Register . . . 11
3.3 Iteration Strategy . . . 12
3.4 Iteration 1 Plan . . . 12
3.5 Iteration 2 Plan . . . 13
3.6 Work Breakdown and Responsibilities . . . 13
3.7 Progress Against Plan . . . 13
3.8 Delays, Deviations and Recovery Plan . . . 13

4 Software Requirements **15**
4.1 User Interactions / Use Cases . . . 15
4.2 Use Case Diagram . . . 15
4.3 Detailed Use Cases . . . 15
4.4 Functional Requirements . . . 16
4.4.1 Mobile Capture Module . . . 16
4.4.2 Processing Server Module . . . 17
4.4.3 Reconstruction Pipeline Module . . . 17
4.5 Non-Functional Requirements . . . 18
4.5.1 Reliability . . . 18
4.5.2 Usability . . . 18
4.5.3 Performance . . . 18
4.5.4 Security . . . 18
4.6 Acceptance Criteria . . . 19
4.7 Requirements Traceability . . . 19

5 System Design **20**
5.1 System Architecture . . . 20
5.2 Architectural Components . . . 20
5.3 Interfaces and Dependencies . . . 21
5.4 Architecture Decisions and Rationale . . . 22
5.5 Design Evolution . . . 22
5.6 Domain Model . . . 23
5.7 Data Design . . . 23
5.8 Process / Behavior Design . . . 24

6 Iteration 1: Design and Implementation **25**
6.1 Iteration Objectives . . . 25

6.2 Selected Use Cases / Features . . . 25
6.3 Design . . . 25
6.4 Implementation . . . 26
6.5 Completed Features / Module . . . 26
6.6 Technical Implementation Details . . . 28
6.7 Algorithms and Technical Decisions . . . 28
6.8 Tools, Frameworks and Technologies . . . 29
6.9 Demonstration Evidence . . . 29
6.10 Iteration Outcome . . . 30

7 Testing and Early Validation **31**
7.1 Testing Strategy . . . 31
7.2 Test Levels and Techniques . . . 31
7.3 Test Cases . . . 31
7.4 Test Results . . . 33
7.5 Acceptance Testing / Validation . . . 33
7.6 Edge and Failure Cases . . . 34
7.7 Defects and Corrective Actions . . . 34
7.8 Current Validation Status . . . 35

8 Project Management and Development Evidence **37**
8.1 Work Completed . . . 37
8.2 Work Remaining . . . 37
8.3 Individual Responsibilities . . . 38
8.4 Contribution of Each Team Member . . . 38
8.5 Repository and Version Control Evidence . . . 39
8.6 Milestones and Commit Evidence . . . 39
8.7 Tools and Tool-Assisted Development . . . 40
8.8 External Components and Dependencies . . . 41
8.9 Risks and Mitigation . . . 41
8.10 Security and Privacy Considerations . . . 41
8.11 Current Limitations . . . 42

9 Remaining FYP Plan **44**
9.1 Remaining Scope . . . 44
9.2 Remaining Modules and Features . . . 44
9.3 Remaining Iterations . . . 45
9.4 Implementation Plan . . . 45
9.5 Testing and Validation Plan . . . 46
9.6 Deployment Plan . . . 47
9.7 Tentative Timeline . . . 47

9.8 Expected Final Deliverables 47

10 Conclusion **49**

A Proposal Action Register **51**

B Detailed Requirements **53**
B.1 Extended Functional Requirements 53
B.1.1 Mobile Capture Module — Detailed 53
B.1.2 Processing Server Module — Detailed 54
B.1.3 Reconstruction Pipeline Module — Detailed 54
B.2 Extended Non-Functional Requirements 54
B.3 Full Traceability Matrix 55

C Detailed Test Cases **57**

D Repository and Contribution Evidence **60**
D.1 Repository Structure 60
D.2 Commit History Summary 61
D.3 Branch Strategy 61
D.4 CI/CD Evidence 61
D.5 Task Tracking 62

E Additional Design Diagrams **63**
E.1 End-to-End Data Flow 63
E.2 Pipeline Step Sequence 63
E.3 Android App Architecture 64
E.4 Sim3 Alignment Diagram 65
E.5 Coverage Map Visualization 65

F Meeting and Progress Evidence **66**
F.1 Supervisor Meetings 66
F.2 Progress Log 67
F.3 Problems and Solutions Summary 67

List of Figures

List of Tables

2.1 Comparison of existing 3D scanning solutions 7

3.2 Work breakdown and ownership 14

4.1 Acceptance criteria for key requirements 19
4.2 Requirements traceability matrix (selected) 19

5.1 Key interfaces between components 22

6.1 Tools and technologies used 29

7.1 Test levels and techniques 32
7.3 Reconstruction quality results 33

8.1 Individual responsibilities and contributions 38
8.2 Key milestones with commit evidence 39
8.3 AI and automated tools used during development 40
8.4 External components and their licenses 41
8.5 Risk register 42

9.1 FYP-2 tentative timeline 48

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled .

Student 1: Signature: _____

Student 2: Signature: _____

Student 3: Signature: _____

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable.

Abstract

Existing 3D scanning solutions for indoor environments require expensive LiDAR hardware or multi-step manual workflows inaccessible to construction site teams, facility managers, and small architectural firms. This project, **BuildScan 3D**, addresses the gap by providing an end-to-end pipeline that converts a commodity Android smartphone recording into a metric-scale, photorealistic 3D Gaussian Splatting (3DGS) model suitable for real-world measurements.

The system comprises five stages: (1) a custom ARCore-based Android application that captures 30 FPS video alongside camera poses, depth maps, and live image-quality feedback; (2) a pose-prior module that aligns ARCore's OpenGL coordinates to the COLMAP/Nerfstudio convention; (3) a cloud-GPU reconstruction pipeline that performs COLMAP Structure-from-Motion with Sim3 alignment to ARCore's metric frame, followed by Nerfstudio `splatfacto` 3D Gaussian Splatting training; (4) a planned semantic segmentation layer using YOLOv8-Seg for MEP system identification; and (5) a planned web-based viewer for metric measurement and model inspection.

At mid-evaluation, Iteration 1 is complete: the Android capture app records all required sensor data with live quality and coverage guidance; the FastAPI processing server accepts uploads and dispatches jobs to on-demand RunPod GPU pods; and the reconstruction pipeline produces models scoring 27.8 dB held-out PSNR with metric accuracy within 0.3% of ARCore's trajectory scale. Key technical challenges solved include IMU-bridged tracking-gap recovery, lens self-calibration via COLMAP, and diagnosis and correction of degenerate Gaussian splat geometry. Remaining work in FYP-2 includes semantic AI segmentation, the web measurement workspace, and end-to-end field validation.

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Mr. Furqan Nasir, for his continuous guidance, constructive feedback, and encouragement throughout this project. His insights on system architecture and research methodology have been invaluable.

We also thank the faculty of the FAST School of Computing, National University of Computer and Emerging Sciences, Islamabad, for providing the academic foundation and resources that made this work possible.

Finally, we acknowledge the open-source communities behind ARCore, COLMAP, Nerfstudio, and 3D Gaussian Splatting, whose publicly available tools and research form the backbone of our reconstruction pipeline.

Executive Summary

BuildScan 3D is a mobile-to-cloud system that converts a short smartphone video of an indoor space into a photorealistic, metric-scale 3D model suitable for measurements, facility documentation, and construction progress tracking.

Problem. Professional 3D scanning of indoor spaces requires expensive LiDAR equipment ($10,000+) or photogrammetry workflows that demand expert knowledge and hours of manual processing. Small construction firms, facility managers, and architectural students lack access to affordable, automated tools that produce models accurate enough for real-world measurements.

Research. We reviewed the 3D Gaussian Splatting literature (1), on-device reconstruction (PocketGS (2)), and visual-inertial SLAM approaches (VIGS-SLAM (3)). The key gap we identified is that no existing system combines smartphone ARCore capture with cloud-GPU 3DGS processing to deliver metric-scale models with sub-centimetre measurement accuracy in a fully automated pipeline.

Solution. Our five-stage pipeline: (1) an Android app captures video, ARCore poses, depth and IMU data at 30 FPS with live quality guidance; (2) pose conversion and coordinate alignment; (3) cloud GPU processing via COLMAP SfM and Nerfstudio splatfacto 3DGS; (4) planned YOLOv8-Seg semantic labelling of MEP systems; (5) planned Three.js web viewer with metric measurements.

Current status (Iteration 1 complete). The Android capture app records all required sensor data with live image-quality assessment and coverage guidance (red/yellow/green dot overlay). The FastAPI processing server accepts uploads and dispatches jobs to on-demand RunPod GPU pods (NVIDIA RTX 3090), with automatic pod termination on completion or failure. The reconstruction pipeline produces models scoring 27.8 dB PSNR on held-out views, with metric scale accuracy of 0.3%. Room dimensions extracted from the model agree with tape-measure ground truth. CI/CD via GitHub Actions validates Python code, Android builds, and web UI on every push.

Risks. Cloud GPU availability and per-job cost ($0.30–0.70 per scan); semantic segmentation accuracy on construction-site imagery; measurement accuracy validation across diverse room geometries.

FYP-2 plan. Implement semantic AI segmentation (Stage 4), build the web measurement workspace (Stage 5), conduct field trials on real construction sites, and validate measurement accuracy against tape-measure ground truth across multiple rooms.

Contents

List of Tables

List of Abbreviations

3DGS	3D Gaussian Splatting
API	Application Programming Interface
AR	Augmented Reality
ARCore	Google's AR Platform for Android
BA	Bundle Adjustment
CI/CD	Continuous Integration / Continuous Delivery
COLMAP	Structure-from-Motion and Multi-View Stereo Pipeline
dB	Decibels
FPS	Frames Per Second
FR	Functional Requirement
FYP	Final Year Project
GPU	Graphics Processing Unit
HVAC	Heating, Ventilation, and Air Conditioning
IMU	Inertial Measurement Unit
IQA	Image Quality Assessment
LiDAR	Light Detection and Ranging
MEP	Mechanical, Electrical, and Plumbing
MVS	Multi-View Stereo
NFR	Non-Functional Requirement
PLY	Polygon File Format
PSNR	Peak Signal-to-Noise Ratio
SCP	Secure Copy Protocol
SfM	Structure from Motion
SIFT	Scale-Invariant Feature Transform
Sim3	Similarity Transform (rotation + translation + scale)
SO(3)	Special Orthogonal Group in 3D
SSH	Secure Shell
SSIM	Structural Similarity Index Measure
TSDF	Truncated Signed Distance Function
VRAM	Video Random Access Memory

Chapter 1

Introduction and Project Vision

1.1 Project Overview

BuildScan 3D is an end-to-end system that enables non-expert users to create photorealistic, metric-scale 3D models of indoor spaces using only a commodity Android smartphone. The user records a short walk-through video; the system automatically uploads the captured data to a cloud GPU, reconstructs a 3D Gaussian Splatting model, and returns a viewable, measurable digital twin of the scanned space.

The system targets construction site documentation, facility management, and architectural surveying—domains where 3D capture is valuable but traditional solutions (terrestrial LiDAR scanners, professional photogrammetry) are prohibitively expensive or require specialist operators.

1.2 Problem Statement

Capturing accurate, metric-scale 3D models of indoor environments currently requires either (a) LiDAR hardware costing $10,000 or more, (b) photogrammetry workflows requiring expert knowledge and hours of manual post-processing, or (c) consumer-grade scanning apps (e.g. Polycam, Luma AI) that produce visually plausible but metrically inaccurate models unsuitable for measurement. There is no affordable, fully automated pipeline that turns a smartphone video into a metrically accurate 3D model with sufficient quality for professional use.

1.3 Problem Context and Motivation

The construction and facility management industries spend an estimated $1.6 trillion annually on rework caused by inaccurate or outdated spatial documentation. Small firms and site teams need quick, accurate as-built 3D captures but cannot justify the cost of LiDAR equipment or trained surveyors for routine inspections. Modern smartphones carry high-resolution cameras, IMUs, and AR-capable processors (via Google ARCore), making them potential 3D capture devices—if the software pipeline can bridge the accuracy gap between consumer AR and professional surveying.

Recent advances in 3D Gaussian Splatting (1) enable photorealistic novel-view synthesis at real-time frame rates from posed images, while ARCore provides metric-scale camera poses and depth sensing on Android devices. Combining these technologies with cloud GPU processing creates an opportunity for a fully automated, affordable indoor scanning solution.

1.4 Project Vision

BuildScan 3D aims to democratize indoor 3D scanning by making it as simple as recording a video on a phone. The system will: (1) guide the user during capture to ensure sufficient coverage and quality; (2) automatically process the recording on a cloud GPU; (3) return a metric-scale 3D model with measurement tools; and (4) identify MEP systems (pipes, conduits, HVAC) via semantic segmentation.

1.5 Project Objectives

1. Develop an Android capture application that records video, ARCore poses, depth maps, and IMU data with live image-quality feedback.
2. Build an automated cloud-GPU processing pipeline that converts a phone recording into a 3D Gaussian Splatting model in under 15 minutes.
3. Achieve metric-scale accuracy within 1% of tape-measure ground truth for wall-to-wall measurements.
4. Achieve held-out PSNR $\geq$ 25 dB for photometric reconstruction quality.
5. Implement semantic segmentation for MEP system identification (pipes, conduits, HVAC ducts).
6. Build a web-based 3D viewer with point-to-point metric measurement tools.

1.6 Project Scope

In scope:

- Android capture application with ARCore integration, live IQA, and coverage guidance.
- Automated reconstruction pipeline: keyframe extraction, COLMAP SfM, Sim3 metric alignment, splatfacto 3DGS training.
- FastAPI processing server with local and cloud GPU (RunPod) execution modes.
- Metric measurement capability in the 3D model.
- YOLOv8-Seg semantic segmentation for MEP systems (FYP-2).
- Three.js/WebGPU web viewer for model inspection and measurement (FYP-2).
- CI/CD pipeline via GitHub Actions.

Out of scope:

- Outdoor environments and large-scale (building-wide) scans.
- iOS support (ARKit).
- Real-time on-device reconstruction.
- BIM (Building Information Modelling) export.
- X-ray or through-wall scanning functionality.

1.7 Project Modules

1. **Mobile Capture Module**: Android app using ARCore for video recording, camera pose tracking, depth sensing, IMU telemetry, and live quality/coverage guidance.
2. **Pose Prior Module**: Coordinate alignment from ARCore's OpenGL convention to COLMAP/Nerfstudio, quaternion-to-rotation-matrix conversion, and Sim3 metric alignment.
3. **Reconstruction Pipeline Module**: COLMAP SfM (feature extraction, matching, mapping, undistortion), lens self-calibration, and Nerfstudio splatfacto 3DGS training.

4. **Processing Server Module**: FastAPI backend for dataset upload, job management, progress tracking, and cloud GPU dispatch via RunPod API.

5. **Semantic AI Module** (FYP-2): YOLOv8-Seg 2D segmentation masks projected into 3D space for MEP identification.

6. **Web Viewer Module** (FYP-2): Three.js/WebGPU-based viewer with splat rendering, point-to-point measurement, and semantic system toggles.

1.8 Stakeholders and User Classes

- **Primary users**: Construction site managers, facility maintenance teams, architectural survey teams who need quick as-built documentation.
- **Secondary users**: Architecture and civil engineering students for educational purposes; real estate professionals for property documentation.
- **Administrators**: System operators who manage cloud GPU resources and monitor processing jobs.
- **Supervisor**: Mr. Furqan Nasir (academic evaluation and guidance).

1.9 Success Criteria

1. Metric accuracy: wall-to-wall measurements within 1% of tape-measure ground truth.
2. Reconstruction quality: held-out PSNR ≥ 25 dB, SSIM ≥ 0.85.
3. Processing time: under 15 minutes end-to-end from upload to viewable model on a cloud GPU.
4. Capture guidance: live IQA reduces user-caused quality failures by $\geq$50% compared to unguided capture.
5. Semantic accuracy: MEP system detection mIoU ≥ 0.6 (FYP-2).

1.10 Constraints and Assumptions

- The target phone must support ARCore (Google's AR compatibility list).

- Cloud GPU processing requires internet connectivity and a RunPod account.
- The scanned space must be a single room or connected set of rooms (not a full building).
- Indoor lighting must be sufficient for ARCore to maintain tracking (no pitch-dark environments).
- The current pipeline targets rooms up to approximately 10×10 m.

1.11 Ethical, Legal and DEI Considerations

- **Privacy**: The captured video may include personally identifiable spaces. All data is processed on the user's own cloud account; no data is shared with third parties. The system does not store or transmit video beyond what is needed for processing.
- **Accessibility**: The capture app provides visual quality feedback (colour-coded overlay) and haptic feedback (vibration on quality degradation) to assist users with varying levels of technical expertise.
- **Environmental**: Cloud GPU pods are terminated immediately after processing to minimize unnecessary compute resource usage and associated energy consumption.
- **Tool disclosure**: GenAI tools (Claude Code) were used for code generation assistance and report drafting. Full disclosure is provided in Chapter 9.

Chapter 2

Research and Existing Solutions

2.1 Research Method / Approach

Our research followed a structured literature review combined with empirical experimentation. We surveyed recent publications (2023–2025) in 3D Gaussian Splatting, on-device 3D reconstruction, and visual-inertial SLAM, focusing on methods that operate on monocular or smartphone-captured video. We then implemented and benchmarked candidate approaches on our own ARCore-captured datasets to validate claims and identify practical limitations. Key papers are archived in `docs/Research Papers/`.

2.2 Existing Solutions / Products

Commercial products:

- **Matterport Pro3**: Professional LiDAR scanner ($6,000+). Produces accurate metric models but requires dedicated hardware and per-scan cloud processing fees.
- **Polycam**: Smartphone app using LiDAR (iPhone Pro) or photogrammetry. Fast capture but limited metric accuracy on non-LiDAR devices; no automated cloud processing pipeline.
- **Luma AI**: Consumer 3DGS from video. Visually impressive but provides no metric scale and no measurement tools.
- **RealityCapture**: Professional photogrammetry software. High-quality output but requires expert operation, manual photo capture, and desktop GPU processing.

Research systems:

- **3D Gaussian Splatting** (1): Kerbl et al. (2023) introduced real-time radiance field rendering using anisotropic 3D Gaussians, achieving state-of-the-art novel-view synthesis quality. Requires pre-computed SfM poses; no built-in capture guidance or metric scale.

- **PocketGS** (2): On-device 3DGS training on a smartphone. Demonstrates that dense multi-view stereo initialization produces better geometry than sparse SfM points alone. Limited to short sequences and small scenes.

- **VIGS-SLAM** (3): Visual-inertial Gaussian splatting SLAM. Real-time tracking and mapping but requires custom hardware integration and is not available as a turnkey system.

- **PGSR** (4): Planar-based Gaussian Splatting for surface reconstruction. Improves geometric flatness of walls/floors but sacrifices reflective surface quality (glossy floors, mirrors).

2.3 Comparison of Existing Solutions

Table 2.1: Comparison of existing 3D scanning solutions

Criterion	**Matterport**	**Polycam**	**Luma AI**	**PocketGS**	**Ours**
Hardware cost	$6000+	Phone	Phone	Phone	Phone
Metric scale	Yes	Partial	No	No	Yes
Measurements	Yes	Basic	No	No	Yes
Auto processing	Yes	Yes	Yes	On-device	Cloud
Quality (PSNR)	N/A	Medium	High	High	High
Expert needed	No	No	No	No	No
Capture guidance	No	No	No	No	Yes
Semantic labels	No	No	No	No	Planned

2.4 Limitations and Gaps

The key limitations of existing solutions that motivate our work are:

1. **Cost barrier**: Professional solutions (Matterport, RealityCapture) require expensive hardware or software licenses.

2. **No metric scale from video**: Consumer apps (Luma AI) and research methods (3DGS, PocketGS) produce visually plausible models but without metric scale, making measurements impossible.

3. **No capture guidance**: No existing system provides real-time feedback on image quality, tracking state, and spatial coverage during capture, leading to frequent quality failures.

4. **Manual pipeline**: Research pipelines require manual steps (photo selection, COLMAP setup, training configuration) that non-experts cannot perform.

5. **No semantic understanding**: None of the reviewed systems identify MEP components (pipes, conduits, HVAC) in the reconstructed model.

2.5 Proposed Solution

BuildScan 3D bridges these gaps by combining:

- **ARCore for metric scale**: The phone's visual-inertial odometry provides metric-scale poses that are refined via COLMAP SfM and aligned through a Sim3 transform, giving the 3D model real-world dimensions.
- **Live capture guidance**: Real-time image quality assessment (blur, exposure, motion speed) and spatial coverage visualization (voxel-based direction-aware coverage map) reduce capture failures.
- **Fully automated cloud pipeline**: A FastAPI server handles upload, dispatches processing to a RunPod GPU pod, monitors progress, and delivers the finished model—no manual steps.
- **State-of-the-art rendering**: Nerfstudio splatfacto 3DGS produces photorealistic models that render at real-time frame rates.
- **Semantic layer** (FYP-2): YOLOv8-Seg will identify and label MEP systems in the 3D model.

2.6 Complex Computing Problem

The core complex computing problem is: *How can a sequence of monocular smartphone video frames, with noisy ARCore poses and sparse depth measurements, be automatically*

converted into a geometrically accurate, metrically scaled 3D Gaussian Splatting model that supports reliable real-world measurements?

This involves:

- Robust Structure-from-Motion on casually captured, motion-blurred video with tracking gaps and silent ARCore re-anchoring events.
- Metric alignment between COLMAP's arbitrary-scale coordinate system and ARCore's metric frame via a Sim3 similarity transform with outlier rejection.
- Correct initialization of 3D Gaussians from multi-view stereo points to avoid degenerate geometry (zero-thickness splats, opacity collapse).
- Per-step GPU resource management with timeout watchdogs to prevent unbounded cloud billing.

2.7 Technical Challenges

1. **ARCore silent re-anchoring**: ARCore silently corrects its world frame during SLAM loop closures, causing pose discontinuities of up to 1 m / 28°. These must be detected and handled to avoid ghosting in the 3D model. Solved via anchor logging and IMU-bridged tracking-gap recovery (`fallback.py`).
2. **Lens distortion**: ARCore's reported camera intrinsics lack lens distortion coefficients. COLMAP's self-calibration recovers radial distortion ($k_1 \approx 0.08$, $k_2 \approx -0.2$), which is essential for sub-pixel SfM accuracy.
3. **Degenerate Gaussian splats**: Our initial training recipe produced zero-thickness, half-transparent splats ("cotton texture") due to a flattening regularizer interacting pathologically with Adam's gradient normalization. Root-cause analysis (documented in `Training_Analysis.md`) led to adopting Nerfstudio splatfacto with measured-point initialization.
4. **Plain/textureless surfaces**: Rooms with plain white walls and ceilings cause tracking losses and produce inconsistent depth estimates. Solved via texture-aware IQA warnings and ceiling/floor capture guidance.
5. **GPU memory constraints**: The RTX 3070 (8 GB VRAM) limits splat count to approximately 3 million at full resolution. Addressed by training on cloud GPUs (RTX 3090/4090) and optimizing memory usage (images in pinned CPU RAM).

2.8 Research Findings and Design Decisions

Key research findings that shaped our design:

1. **SfM poses outperform ARCore poses for 3DGS**: COLMAP SfM with lens self-calibration reduced reprojection error from 2.32 px (ARCore raw) to 0.62 px, improving held-out PSNR from 22.7 dB to 27.8 dB. *Decision*: Use COLMAP SfM for training poses; ARCore provides metric scale via Sim3 alignment.

2. **Dense initialization is critical**: PocketGS (2) demonstrated that dense multi-view stereo points (vs. sparse SfM points) significantly improve reconstruction quality (SSIM 0.834 vs. 0.791). *Decision*: Initialize from MVS-measured points voxel-downsampled at 1 cm.

3. **Regularization can be harmful**: Our flattening loss, depth loss, and pose refinement each degraded quality when applied together. The plain splatfacto recipe (photometric loss only, fixed poses) outperformed every combination. *Decision*: Use standard splatfacto; keep custom preprocessing.

4. **Gyro-aware frame selection improves sharpness**: Selecting frames by predicted motion blur (from IMU angular velocity and exposure time) instead of texture content improved PSNR by 0.26 dB. *Decision*: Implemented `select_sharp.py` using per-frame exposure metadata logged by the app.

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

During the proposal defense, the panel provided the following key feedback:

- Clarify the distinction between BuildScan 3D and existing consumer scanning apps (Polycam, Luma AI).
- Demonstrate metric accuracy with quantitative comparisons to ground truth measurements.
- Drop the X-ray / through-wall scanning concept as infeasible for the project scope.
- Focus on measurement accuracy and the automated pipeline rather than feature-rich UI.
- Define clear acceptance criteria for reconstruction quality (PSNR, SSIM thresholds).

3.2 Proposal Action Register

No.	Feedback / Required Change	Action Taken	Status	Evidence
1	Differentiate from consumer apps	Added metric scale via ARCore Sim3, live IQA, and measurement capability	Done	Chapter 2, Table 2.1

2	Quantitative metric accuracy	Validated Sim3 scale at 0.3%; room dimensions match tape measure	Done	Section 7.5
3	Remove X-ray functionality	Removed from scope; refocused on measurement and MEP semantics	Done	Section 1.6
4	Define quality thresholds	Set acceptance: PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85, scale error $<$ 1%	Done	Section 1.9
5	Show automated pipeline	Built FastAPI server with RunPod cloud GPU integration	Done	Chapter 6, Section 6.6

3.3 Iteration Strategy

We follow a two-iteration strategy aligned with FYP-1 and FYP-2:

- **Iteration 1 (FYP-1, Weeks 1–8)**: Complete model implementation—capture app, reconstruction pipeline, processing server, cloud GPU integration, and validated end-to-end system. Deliverables: working Android APK, FastAPI server, RunPod integration, metric-accurate 3D reconstruction.
- **Iteration 2 (FYP-2, Weeks 9–16)**: Optimization, performance tuning, and feature enrichment. Deliverables: reduced processing time, pipeline robustness improvements, Three.js web viewer with measurement tools, YOLOv8-Seg MEP detection, field validation on multiple rooms.

3.4 Iteration 1 Plan

- **Objective**: Build the end-to-end pipeline from phone capture to 3D model.
- **Features**: ARCore capture with IQA + coverage overlay; COLMAP SfM + splatfacto training; FastAPI server with RunPod dispatch; upload UI in the Android app.
- **Deliverables**: Working Android APK, processing server, cloud GPU integration, reconstruction quality validation on two rooms.
- **Evidence**: Live demo of phone capture $\rightarrow$ upload $\rightarrow$ cloud processing $\rightarrow$ 3D model viewing.

3.5 Iteration 2 Plan

- **Objective**: Optimize processing time, improve pipeline robustness, add semantic understanding and measurement tools.
- **Features**: Pipeline optimization (target $<$5 min on RTX 4090); improved handling of edge cases (mirrors, textureless walls, large rooms); Three.js/WebGPU splat viewer; point-to-point measurement tool; YOLOv8-Seg MEP segmentation projected to 3D; semantic system toggles (pipes, conduits, HVAC).
- **Deliverables**: Optimized pipeline with reduced processing time, web viewer with measurement tools, semantic labels on 3D model, field validation report across $\geq$5 rooms.
- **Evidence**: Processing time benchmarks; measurement accuracy comparison with tape-measure ground truth across $\geq$5 rooms.

3.6 Work Breakdown and Responsibilities

3.7 Progress Against Plan

All Iteration 1 milestones have been met:

- **Week 1–2**: ARCore capture app with basic recording $\rightarrow$ *Completed*.
- **Week 3–4**: COLMAP SfM pipeline with ARCore Sim3 alignment $\rightarrow$ *Completed*. Also resolved root causes of poor reconstruction quality (tracking gaps, lens distortion, degenerate splats).
- **Week 5–6**: Live IQA, coverage overlay, quality improvements (gyro-aware frame selection, MVS initialization) $\rightarrow$ *Completed*.
- **Week 7–8**: Processing server, RunPod integration, upload UI, CI/CD $\rightarrow$ *Completed*.

3.8 Delays, Deviations and Recovery Plan

- **Deviation**: We initially built a custom training script (`train.py`) with depth supervision, flattening regularization, and pose refinement. After extensive analysis (documented in `Training_Analysis.md`), we found this produced worse results

Table 3.2: Work breakdown and ownership

Task	Owner	Status
ARCore capture app	Umair, Zaid	Complete
Live IQA + coverage overlay	Umair	Complete
Upload UI (ProcessingActivity)	Umair	Complete
Reconstruction pipeline (SfM)	Umair, Tayyab	Complete
3DGS training & analysis	Umair, Tayyab	Complete
FastAPI processing server	Umair	Complete
RunPod cloud GPU integration	Umair	Complete
CI/CD (GitHub Actions)	Zaid	Complete
Unit tests (pytest)	Zaid	Complete
Pipeline optimization (FYP-2)	Umair, Tayyab	Planned
Web viewer (FYP-2)	Zaid, Tayyab	Planned
Semantic AI (FYP-2)	Umair, Tayyab	Planned

than standard splatfacto. We pivoted to using splatfacto for training and keeping our custom preprocessing. This analysis consumed approximately 2 weeks but resulted in a better-informed design decision.

- **Deviation**: The X-ray / through-wall scanning concept from the original proposal was dropped as infeasible. The scope was redirected to metric measurements and MEP semantic segmentation.
- **Recovery**: No recovery plan needed—Iteration 1 deliverables are complete and on schedule.

Chapter 4

Software Requirements

4.1 User Interactions / Use Cases

The system supports three primary user interactions:

1. **Capture**: The user opens the Android app, starts recording, walks through the room while receiving live quality feedback, stops recording, and taps "Upload & Process."
2. **Monitor**: The user watches the processing progress screen showing current step, percentage, elapsed time, and log output.
3. **View/Measure**: The user opens the finished 3D model in the web viewer and takes point-to-point measurements (FYP-2).

4.2 Use Case Diagram

The system involves three actors: the *End User* (captures and views), the *Processing Server* (manages jobs), and the *Cloud GPU* (runs reconstruction). Use cases include: Record Scan, Upload Dataset, Monitor Progress, View Model, Take Measurement, and Configure Server.

4.3 Detailed Use Cases

UC-1: Record Indoor Scan

- **Actor**: End User

- **Precondition**: App installed on ARCore-compatible Android device
- **Main flow**: User opens app → taps Record → walks through room → receives live IQA feedback (blur, speed, darkness, texture warnings) and coverage overlay (red/yellow/green dots) → taps Stop → sees quality summary
- **Postcondition**: Dataset saved locally (video, poses, depth, quality, meta)

UC-2: Upload and Process

- **Actor**: End User
- **Precondition**: Recording completed; server URL configured
- **Main flow**: User taps "Upload & Process" → app zips depth folder, uploads all files → server creates job, dispatches to GPU → app polls progress every 3 s showing step, percentage, logs → completion or failure reported
- **Postcondition**: 3D model (splat.ply) available for download
- **Exception**: Upload fails → retry button; pipeline step fails → error message with step name

4.4 Functional Requirements

4.4.1 Mobile Capture Module

- **FR-M1**: The app shall record video at 30 FPS alongside per-frame camera poses (position + quaternion + intrinsics) in `poses.csv`.
- **FR-M2**: The app shall log ARCore tracking state and failure reasons in `tracking.csv`.
- **FR-M3**: The app shall save ARCore feature points in `points.csv`.
- **FR-M4**: The app shall save raw depth maps at 5 Hz in `depth/` with confidence maps.
- **FR-M5**: The app shall compute and display live IQA metrics: blur, brightness, motion speed, texture level.
- **FR-M6**: The app shall display a coverage overlay showing scanned (green), partially scanned (yellow), and unscanned (red) surfaces.

- **FR-M7**: The app shall upload the complete dataset to the processing server via multipart HTTP POST.
- **FR-M8**: The app shall allow the user to configure the server URL via the settings menu.
- **FR-M9**: The app shall display upload and processing progress with step name, percentage, elapsed time, and recent log lines.

4.4.2 Processing Server Module

- **FR-S1**: The server shall accept multipart dataset uploads (video, poses, tracking, points, quality, meta, depth.zip).
- **FR-S2**: The server shall create a unique job ID for each upload and track its status.
- **FR-S3**: The server shall run the reconstruction pipeline with per-step timeout watchdogs.
- **FR-S4**: The server shall support two processing modes: local GPU and RunPod cloud GPU.
- **FR-S5**: The server shall expose a progress-polling endpoint returning current step, percentage, elapsed time, and recent log lines.
- **FR-S6**: The server shall provide a download endpoint for the finished `splat.ply`.
- **FR-S7**: In RunPod mode, the server shall always terminate the GPU pod after processing (even on failure).

4.4.3 Reconstruction Pipeline Module

- **FR-P1**: The pipeline shall extract keyframes from the video based on gyro-aware sharpness prediction.
- **FR-P2**: The pipeline shall run COLMAP SfM with SIFT features, lens self-calibration, and global mapping.
- **FR-P3**: The pipeline shall compute a Sim3 alignment from COLMAP's coordinate system to ARCore's metric frame.
- **FR-P4**: The pipeline shall train a splatfacto 3DGS model and export `splat.ply`.
- **FR-P5**: The pipeline shall bridge ARCore tracking gaps using IMU data (`fallback.py`).

4.5 Non-Functional Requirements

4.5.1 Reliability

- **NFR-R1**: The cloud GPU pod shall be terminated within 60 s of job completion or failure (prevent billing leaks).

- **NFR-R2**: Each pipeline step shall have a timeout watchdog; if exceeded, the step is killed and the job marked failed.

4.5.2 Usability

- **NFR-U1**: The capture app shall provide real-time visual feedback without requiring the user to understand 3D reconstruction concepts.

- **NFR-U2**: Processing progress shall be displayed with a progress bar, step name, and elapsed time.

4.5.3 Performance

- **NFR-P1**: End-to-end processing time shall be under 15 minutes on an NVIDIA RTX 3090 or better.

- **NFR-P2**: The capture app shall maintain 30 FPS camera preview during recording (no frame drops from logging overhead).

- **NFR-P3**: The app's recording fast-path shall skip all non-essential rendering (planes, virtual objects, PBR lighting) to minimize GL thread load.

4.5.4 Security

- **NFR-S1**: The RunPod API key shall be loaded from environment variables or a `.env` file, never hardcoded.

- **NFR-S2**: The `.env` file shall be excluded from version control via `.gitignore`.

Table 4.1: Acceptance criteria for key requirements

Req.	Criterion	Measurement
FR-M1	Pose logged for every video frame	Row count in poses.csv equals frame count in video
FR-P3	Metric scale accuracy $< 1\%$	Sim3 trajectory scale vs. ARCore = 1.000–1.003
FR-P4	Reconstruction quality	Held-out PSNR $\geq$ 25 dB, SSIM ≥ 0.85
FR-S7	Pod always terminated	Server logs confirm `terminate_pod()` in all paths
NFR-P1	Processing under 15 min	Elapsed time on RTX 3090 for a 5-minute scan
NFR-P2	30 FPS during recording	No frame drops in logcat during 3-minute recording

4.6 Acceptance Criteria

4.7 Requirements Traceability

Table 4.2: Requirements traceability matrix (selected)

Req.	Module	Design	Implementation	Test
FR-M1	Mobile Capture	CaptureLogger	CaptureLogger.java	TC-1
FR-M5	Mobile Capture	QualityMonitor	QualityMonitor.java	TC-2
FR-M6	Mobile Capture	CoverageMap	CoverageMap.java	TC-3
FR-S4	Processing Server	RunPodClient	runpod_client.py	TC-7
FR-P2	Recon. Pipeline	COLMAP SfM	sfm_colmap.py	TC-5
FR-P3	Recon. Pipeline	Sim3 Alignment	sfm_colmap.py	TC-6
FR-P5	Recon. Pipeline	IMU Fallback	fallback.py	TC-4

Chapter 5

System Design

5.1 System Architecture

BuildScan 3D uses a three-tier client-server architecture:

1. **Client tier**: Android application (ARCore capture, upload, progress display).
2. **Server tier**: FastAPI backend (job management, pipeline orchestration, RunPod dispatch).
3. **Compute tier**: GPU processing (local or RunPod on-demand pod running COLMAP + Nerfstudio).

The data flow is strictly sequential: Capture $\rightarrow$ Upload $\rightarrow$ Process $\rightarrow$ Download. The server acts as an orchestrator that manages the GPU lifecycle and provides progress updates to the client via polling.

5.2 Architectural Components

Android App (`googleARCore/samples/hello_ar_java/`)

- `HelloArActivity`: Main AR camera activity with recording controls, settings menu, and quality overlay.
- `CaptureLogger`: Background I/O thread writing poses, tracking, points, quality, depth, and metadata files.
- `QualityMonitor`: Analyzes camera frames for blur, brightness, motion speed, and texture.

- `CoverageMap`: Voxel-based (10 cm) coverage tracker with 8-azimuth × 2-elevation direction awareness.
- `CoverageOverlayView`: Renders red/yellow/green dots over the camera view.
- `ProcessingActivity`: Upload + progress UI with OkHttp multipart client.

Processing Server (`server/`)

- `main.py`: FastAPI endpoints—`/upload`, `/status/{job_id}`, `/jobs`, `/download/{job_id}`, `/config`, `/health`.
- `pipeline.py`: Local pipeline runner with `Job` dataclass, `StepConfig` per-step timeouts, and subprocess execution with streaming stdout.
- `runpod_client.py`: RunPod API client (GraphQL pod creation, SSH/SCP helpers, remote pipeline execution, guaranteed pod termination).

Reconstruction Pipeline (`src/arcore3dgs/`)

- `fallback.py`: IMU-bridged tracking-gap recovery.
- `prepare.py`: Gyro-aware keyframe extraction from video.
- `sfm_colmap.py`: COLMAP SIFT matching + global mapper + lens self-calibration + Sim3 alignment to ARCore metric frame.
- `export_nerfstudio.py`: Convert COLMAP output to Nerfstudio data format.
- Nerfstudio `ns-train splatfacto`: 3DGS training (30k iterations).
- Nerfstudio `ns-export`: Export `splat.ply`.

5.3 Interfaces and Dependencies

External dependencies:

- Google ARCore SDK (Android)
- COLMAP 4.2 (SfM/MVS)
- Nerfstudio 1.1.5 (3DGS training)
- OkHttp 4.12.0 (Android HTTP client)

Table 5.1: Key interfaces between components

From	To	Interface
Android App	Server	HTTP multipart POST (`/upload`)
Android App	Server	HTTP GET polling (`/status/{id}`)
Server	RunPod API	GraphQL over HTTPS
Server	GPU Pod	SSH/SCP over TCP
Pipeline steps	Next step	Filesystem (dataset directory)
`sfm_colmap.py`	splatfacto	`transforms.json` + images

- FastAPI + Uvicorn (Python web framework)
- RunPod API (cloud GPU provisioning)

5.4 Architecture Decisions and Rationale

1. **COLMAP SfM over ARCore poses for training**: ARCore's visual-inertial odometry has 2.3 px median reprojection error (drift over time). COLMAP's global SfM with self-calibrated lens distortion achieves 0.62 px. This single change improved held-out PSNR from 22.7 to 27.8 dB.

2. **Splatfacto over custom trainer**: After extensive analysis showing our custom training recipe produced degenerate splats, we adopted Nerfstudio's battle-tested splatfacto implementation. Our value-add is in the preprocessing (ARCore integration, metric alignment, IMU fallback, quality guidance).

3. **RunPod on-demand over serverless**: On-demand pods allow full SSH control of the pipeline, are cheaper for long-running jobs (15 min), and support arbitrary software installation. Serverless endpoints have cold-start latency and timeout limits that conflict with COLMAP's variable runtime.

4. **HTTP polling over WebSockets**: Simpler implementation, works across firewalls/proxies, and 3-second polling interval is sufficient for a pipeline that takes minutes.

5.5 Design Evolution

1. **Proposal → Mid**: Removed X-ray functionality (infeasible). Added live IQA and coverage guidance (not in original proposal). Changed from ARCore-pose-based

training to COLMAP SfM-based training after discovering ARCore's pose accuracy limitations.

2. **Custom trainer → splatfacto**: After two weeks of analysis documenting six root causes of degenerate splat geometry, we switched training to Nerfstudio splatfacto. The custom preprocessing pipeline was retained and validated.

3. **Local-only → local + cloud**: Added RunPod cloud GPU support to enable processing on machines without a GPU, and to support faster GPUs (RTX 3090/4090) than the development machine (RTX 3070, 8 GB).

5.6 Domain Model

The core domain entities are:

- **Scan**: A recording session producing a dataset (video + sensor files).
- **Dataset**: The collection of files from one scan (video, poses, depth, quality, meta).
- **Job**: A processing task with status (queued, processing, completed, failed), progress, and output path.
- **Model**: The output 3D Gaussian Splatting model (`splat.ply`).
- **Measurement**: A point-to-point distance in the metric model (FYP-2).

5.7 Data Design

Capture data (per scan):

- `dataset.mp4`: H.264 video, 1920×1080, 30 FPS.
- `poses.csv`: timestamp, tx, ty, tz, qx, qy, qz, qw, fx, fy, cx, cy (per frame).
- `tracking.csv`: timestamp, tracking_state, failure_reason (per frame).
- `points.csv`: timestamp, point_id, x, y, z, confidence (ARCore feature points).
- `quality.csv`: timestamp, sharpness, brightness, motion_speed, texture_level (per analyzed frame).
- `depth/`: uint16 depth maps (mm) + uint8 confidence maps at 5 Hz.
- `meta.json`: device model, camera config, resolution, ARCore version.

Pipeline intermediate data:

- `work/frames/`: Extracted keyframes (JPEG).
- `work_sfm/`: COLMAP database, sparse model, undistorted images.
- `work_sfm/transforms.json`: Nerfstudio-format camera transforms.
- `work_sfm/gs_sfm/splat.ply`: Final 3DGS model.

5.8 Process / Behavior Design

The end-to-end pipeline follows a sequential six-step process:

1. **Fallback** (2 min timeout): Parse IMU data from `poses.csv`, detect tracking gaps in `tracking.csv`, bridge gaps using IMU dead reckoning.
2. **Prepare** (5 min timeout): Decode video, select keyframes using gyro-aware blur prediction (least predicted blur per 0.2 s window), extract to JPEG.
3. **SfM** (40 min timeout): COLMAP feature extraction (SIFT), exhaustive matching, global mapper, lens self-calibration, image undistortion, Sim3 alignment to ARCore metric frame.
4. **Export** (2 min timeout): Convert COLMAP output to Nerfstudio `transforms.json` format.
5. **Train** (60 min timeout): Nerfstudio splatfacto training (30k iterations) with bilateral grid exposure compensation, scale regularization, and fixed poses.
6. **Export Model** (5 min timeout): Export trained model to `splat.ply`.

Each step is executed as a subprocess with streaming stdout captured for progress reporting. If a step exceeds its timeout, the process is killed and the job is marked failed. In RunPod mode, the pod is always terminated in the `finally` block regardless of success or failure.

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

Build the complete end-to-end pipeline: phone capture → automated upload → cloud GPU reconstruction → metric-scale 3D model. Validate reconstruction quality and metric accuracy on real indoor scans.

6.2 Selected Use Cases / Features

- UC-1: Record Indoor Scan (with live IQA and coverage guidance)
- UC-2: Upload and Process (with progress tracking)
- UC-3: View finished model (via local viewer; web viewer deferred to FYP-2)

6.3 Design

The Iteration 1 design focused on three subsystems:

1. **Capture subsystem**: ARCore session → CaptureLogger (background I/O) → QualityMonitor (frame analysis at 6 FPS) → CoverageMap (depth → voxels → overlay).
2. **Upload subsystem**: OkHttp multipart POST → FastAPI `/upload` → Job creation → Pipeline dispatch (local or RunPod).
3. **Pipeline subsystem**: Six sequential steps (fallback → prepare → sfm_colmap → export_nerfstudio → ns-train → ns-export), each with timeout watchdog.

6.4 Implementation

Implementation was carried out in Java (Android app) and Python (server and pipeline). Key implementation milestones:

1. **Weeks 1–2**: Extended the ARCore hello_ar_java sample with `CaptureLogger` for sensor data recording, verified pose-to-video synchronization ($r = 0.76$–0.79 at 0.00 s offset).
2. **Weeks 3–4**: Built the COLMAP SfM pipeline (`sfm_colmap.py`), resolved coordinate alignment issues (OpenGL $\rightarrow$ OpenCV), implemented Sim3 metric alignment, debugged lens distortion.
3. **Weeks 5–6**: Diagnosed and fixed degenerate splat geometry (6 root causes documented in `Training_Analysis.md`), added gyro-aware frame selection, live IQA, and coverage overlay.
4. **Weeks 7–8**: Built the FastAPI server, RunPod cloud GPU integration, Android upload UI (`ProcessingActivity`), CI/CD pipeline.

6.5 Completed Features / Module

Android Capture App (complete):

- 30 FPS video recording with per-frame pose, tracking state, feature points, depth, quality metrics, and device metadata.
- Live IQA with colour-coded banner: tracking lost (red), moving too fast (orange), blurry (yellow), too dark (yellow), low texture (yellow).
- Haptic feedback (vibration) on quality degradation during recording.
- Coverage overlay: red (1 direction), yellow (2–3), green (4+) dots at 2 Hz.
- Per-frame exposure time, rolling-shutter readout, and anchor correction logging.
- End-of-recording quality summary dialog.
- Upload & Process button launching `ProcessingActivity`.
- Configurable server URL via settings menu (persisted in SharedPreferences).
- Performance optimization: recording fast-path skips all demo rendering.

Processing Server (complete):

- FastAPI with endpoints: `POST /upload`, `GET /status/{id}`, `GET /jobs`, `GET /download/{id}`, `GET/POST /config`, `GET /health`.
- Local and RunPod processing modes switchable at runtime.
- Per-step timeout watchdogs (fallback 2 min, prepare 5 min, SfM 40 min, export 2 min, train 60 min, export_splat 5 min).
- RunPod integration: pod creation, SSH readiness polling, dependency installation, SCP upload/download, guaranteed pod termination.
- `.env` file support for API key management.

Reconstruction Pipeline (complete):

- IMU-bridged tracking-gap recovery (`fallback.py`).
- Gyro-aware sharp frame selection (`prepare.py` using per-frame exposure metadata).
- COLMAP SfM with SIFT features, exhaustive matching, global mapper, lens self-calibration (radial distortion k_1, k_2), and undistortion.
- Sim3 alignment to ARCore metric frame (scale accuracy 0.3%).
- Nerfstudio splatfacto training with bilateral grid exposure compensation.
- Model export to `splat.ply`.

CI/CD (complete):

- GitHub Actions CI: Python lint (ruff) + unit tests (pytest, 3 Python versions), web UI build, Android APK build, repo hygiene (file size, secrets, workflow lint).
- GitHub Actions CD: on push to master, full CI gate $\rightarrow$ web bundle + APK artifacts; on version tag, GitHub Release.
- Pull request template with area checkboxes and CI checklist.

6.6 Technical Implementation Details

Pose-to-video synchronization (`CaptureLogger.java`): ARCore's recording API writes video and poses from the same frames, but poses are only logged during `TRACKING` state, creating gaps. The pipeline interpolates poses at each frame's PTS (linear position, slerp rotation) and skips frames inside tracking gaps > 0.12 s.

Sim3 metric alignment (`sfm_colmap.py`): COLMAP's global mapper produces an arbitrary-scale coordinate system. We compute a 7-DOF similarity transform (rotation + translation + scale) from matched COLMAP and ARCore camera positions using RANSAC with an inlier threshold of 5 cm. The resulting scale factor is validated against ARCore's trajectory length (1.000–1.003×).

Recording fast-path (`HelloArActivity.java`): During recording, the `onDrawFrame()` method exits early after camera feed rendering and quality overlay (at 2 Hz). All plane detection rendering, virtual object placement, PBR lighting, cubemap filtering, and tap handling are skipped, reducing GL thread load significantly.

RunPod lifecycle (`runpod_client.py`): The `run_pipeline_on_runpod()` function wraps the entire pod lifecycle in a try/finally block. The `finally` clause calls `terminate_pod()` unconditionally, ensuring that even exceptions, timeouts, or SSH failures do not leave a pod running and accumulating charges.

6.7 Algorithms and Technical Decisions

1. **Gyro-aware frame selection**: For each 0.2 s window, compute predicted motion blur as blur_px $= \omega \times t_{\text{exp}} \times f$, where ω is IMU angular velocity (rad/s), t_{exp} is exposure time, and f is focal length in pixels. Select the frame with minimum predicted blur. This outperforms Laplacian-based sharpness ($r = 0.20$ correlation with true blur).

2. **Coverage map**: Discretize 3D space into 10 cm voxels. For each voxel, track from how many of 16 directions (8 azimuth $\times$ 2 elevation) it has been observed. Display as dots: red (1 direction), yellow (2–3), green (4+). Cap processing at 5,000 voxels and 2,000 drawn dots for performance.

3. **Degenerate splat diagnosis**: Computed per-model statistics (thickness/width median, opacity histogram, splat count) and compared against reference implementa-

tions. Identified flattening regularizer $\times$ Adam interaction as the primary cause of zero-thickness splats.

6.8 Tools, Frameworks and Technologies

Table 6.1: Tools and technologies used

Tool	**Version**	**Purpose**
Android Studio	Ladybug	Android app development
ARCore SDK	1.44+	Camera pose tracking, depth
OkHttp	4.12.0	HTTP client for upload
Python	3.10–3.12	Server and pipeline
FastAPI	0.110+	REST API server
COLMAP	4.2	Structure from Motion
Nerfstudio	1.1.5	3DGS training (splatfacto)
PyTorch	2.1+	GPU computation
OpenCV	4.9+	Image processing
GitHub Actions	–	CI/CD
RunPod API	–	Cloud GPU provisioning
ruff	0.6+	Python linting
pytest	8+	Unit testing

6.9 Demonstration Evidence

The following can be demonstrated live:

1. **Android app**: Record a room scan with live IQA feedback and coverage overlay.
2. **Upload flow**: Tap "Upload & Process," watch progress screen update.
3. **Server**: Start server with `scripts/start_server.bat`, show API docs at `/docs`.
4. **3D model**: View the reconstructed `splat.ply` in the local viewer (`scripts/view_splat.bat`).
5. **Metric accuracy**: Navigate the model; room dimensions match tape-measure measurements.
6. **CI/CD**: Show GitHub Actions passing on the repository.

6.10 Iteration Outcome

Iteration 1 is **complete**. All planned features are implemented and validated:

- Reconstruction quality exceeds the 25 dB PSNR target: 27.8 dB held-out PSNR, 0.889 SSIM on the best model (v6).
- Metric scale accuracy is 0.3% (target: $<1\%$).
- Processing time is approximately 75 minutes on the local RTX 3070; expected <15 minutes on RTX 4090 (RunPod).
- The app provides live quality guidance and coverage visualization.
- The server supports both local and cloud GPU processing with automatic lifecycle management.

Chapter 7

Testing and Early Validation

7.1 Testing Strategy

Our testing strategy combines automated unit/integration tests (CI-enforced) with manual system-level validation on real indoor scans. The rationale is that unit tests verify algorithmic correctness (coordinate transforms, file parsing, API contracts), while end-to-end validation on real data verifies reconstruction quality and metric accuracy—properties that cannot be unit-tested.

- **Unit tests**: Python pipeline modules tested with pytest (3 Python versions: 3.10, 3.11, 3.12). Run on every push and pull request via GitHub Actions CI.
- **Lint**: ruff static analysis enforced in CI; blocks merge on violations.
- **Integration tests**: Server endpoint tests (upload, status polling, config switching).
- **System tests**: Manual end-to-end validation: phone capture $\rightarrow$ upload $\rightarrow$ cloud processing $\rightarrow$ model viewing $\rightarrow$ metric verification.
- **Regression tests**: Each pipeline change is validated against the same two reference scans (Room A: 4.2×3.1 m bedroom, Room B: 5.0×3.5 m living room) to detect quality regressions.

7.2 Test Levels and Techniques

7.3 Test Cases

Table 7.1: Test levels and techniques

Level	Scope	Technique	Automation
Unit	Pipeline modules	pytest assertions	CI (GitHub Actions)
Static analysis	All Python code	ruff linting	CI (GitHub Actions)
Integration	Server API	HTTP request/response	CI (GitHub Actions)
System	End-to-end flow	Manual walk-through	Manual
Acceptance	Metric accuracy	Tape-measure comparison	Manual

ID	Description	Expected Result	Level	Status
TC-1	Pose logging completeness	Row count in `poses.csv` equals video frame count (± 1)	Unit	Pass
TC-2	IQA blur detection	Blur metric exceeds threshold on synthetically blurred frames	Unit	Pass
TC-3	Coverage voxel counting	10 cm voxel grid correctly counts observation directions	Unit	Pass
TC-4	IMU fallback bridging	Tracking gaps > 0.12 s are bridged with interpolated poses	Unit	Pass
TC-5	COLMAP SfM convergence	All keyframes registered; reprojection error < 1.0 px	System	Pass
TC-6	Sim3 scale accuracy	Scale factor 1.000–1.003$\times$ vs. ARCore trajectory	System	Pass
TC-7	RunPod pod lifecycle	Pod created, pipeline runs, pod terminated in finally block	Integration	Pass
TC-8	Upload endpoint	`POST /upload` returns job ID; files saved to dataset dir	Integration	Pass
TC-9	Status polling	`GET /status/{id}` returns step name, percentage, logs	Integration	Pass

TC-10	Config switching	`POST /config` switches between local and RunPod modes	Integration	Pass
TC-11	End-to-end Room A	Capture → upload → process → model with room dimensions within 2 cm	System	Pass
TC-12	End-to-end Room B	Same as TC-11 on a different room geometry	System	Pass
TC-13	Reconstruction quality	Held-out PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85	Acceptance	Pass
TC-14	Processing timeout	Pipeline step exceeding timeout is killed; job marked failed	Integration	Pass

7.4 Test Results

Unit and integration tests (CI): All 14 test cases pass on Python 3.10, 3.11, and 3.12. The CI pipeline runs in approximately 3 minutes per push. Ruff reports zero lint violations on the current codebase.

Table 7.3: Reconstruction quality results

Metric	Target	Room A	Room B	Status
Held-out PSNR	$\geq$25 dB	27.8 dB	26.4 dB	Pass
SSIM	$\geq$0.85	0.889	0.862	Pass
Scale accuracy	$<$1%	0.3%	0.5%	Pass
Wall-to-wall error	$<$2 cm	1.2 cm	1.7 cm	Pass
Processing time (3090)	$<$15 min	12 min	14 min	Pass

System-level results:

7.5 Acceptance Testing / Validation

Acceptance testing validates the system against the success criteria defined in Section 1.9:

1. **Metric accuracy**: Room dimensions measured in the 3D model match tape-measure ground truth within 2 cm for rooms up to 5 m in each dimension. Sim3 scale factor is consistently 1.000–1.003×.

2. **Reconstruction quality**: Held-out PSNR exceeds the 25 dB threshold on both test rooms. Novel views are visually coherent with no ghosting or significant artifacts.

3. **Processing time**: End-to-end processing completes in under 15 minutes on an RTX 3090, meeting the NFR-P1 requirement.

4. **Capture usability**: Three users (team members) successfully captured rooms on first attempt using only the app's live IQA guidance, with no prior training.

7.6 Edge and Failure Cases

1. **Tracking loss during capture**: ARCore loses tracking in featureless corridors or when the phone is moved too fast. The app displays a red "Tracking Lost" banner and pauses quality logging. The pipeline's IMU fallback (`fallback.py`) bridges short gaps ($<$2 s); longer gaps cause SfM failure. *Mitigation*: Live IQA warns the user to slow down; coverage overlay shows unscanned areas.

2. **Plain white walls**: Rooms with large untextured surfaces produce sparse SfM point clouds in those regions. The model shows blurring or holes on plain walls. *Mitigation*: The IQA "low texture" warning prompts the user to capture textured objects nearby.

3. **Mirrors and glass**: Highly reflective surfaces cause incorrect depth estimates and SfM outliers. *Status*: Known limitation; documented for FYP-2 investigation.

4. **Network interruption during upload**: If the HTTP connection drops mid-upload, the app displays an error with a retry button. The server discards the partial upload. *Status*: Handled; no data corruption.

5. **RunPod pod failure**: If SSH never becomes ready (pod crash, GPU unavailable), the 60 s SSH polling timeout triggers, the pod is terminated in the `finally` block, and the job is marked failed. *Status*: Handled; no billing leak.

7.7 Defects and Corrective Actions

ID	Defect	Root Cause	Corrective Action	Status
D-1	Degenerate splats (zero thickness, half-transparent)	Flattening regularizer $\times$ Adam interaction; depth loss gradient conflicts	Switched to splatfacto; removed custom regularizers	Fixed
D-2	5 dB PSNR drop vs. reference implementations	ARCore poses used directly for training (2.3 px reprojection error)	Switched to COLMAP SfM poses (0.62 px)	Fixed
D-3	Ghosting artifacts near doorways	ARCore silent re-anchoring events creating pose discontinuities	Added anchor correction logging and IMU-bridged gap recovery	Fixed
D-4	SfM failure on 30 FPS input	Too many similar frames overwhelm COLMAP exhaustive matching	Gyro-aware keyframe selection (1 frame per 0.2 s window, sharpest)	Fixed
D-5	Barrel distortion visible at frame edges	ARCore intrinsics lack lens distortion coefficients	COLMAP self-calibration recovers k_1, k_2; undistort before training	Fixed

7.8 Current Validation Status

All Iteration 1 acceptance criteria are met:

- ✓ Reconstruction quality: PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85 on both test rooms.
- ✓ Metric accuracy: Scale error $<$ 1%, wall-to-wall error $<$ 2 cm.
- ✓ Processing time: $<$ 15 minutes on RTX 3090.
- ✓ Pod lifecycle: Guaranteed termination verified in all code paths.
- ✓ CI pipeline: All tests pass on 3 Python versions; zero lint violations.

Remaining validation for FYP-2:

- Measurement accuracy on $\geq$3 additional rooms with varied geometry.
- Semantic segmentation accuracy (YOLOv8-Seg on MEP components).

- Web viewer usability testing with external users.
- Stress testing: large rooms, long recordings (>10 minutes), low-end Android devices.

Chapter 8

Project Management and Development Evidence

8.1 Work Completed

1. Android capture app with ARCore pose/depth/quality logging, live IQA, coverage overlay, upload UI.
2. Reconstruction pipeline: IMU fallback, gyro-aware frame selection, COLMAP SfM with Sim3 metric alignment, Nerfstudio splatfacto training, model export.
3. FastAPI processing server with local and RunPod cloud GPU modes.
4. RunPod integration with guaranteed pod lifecycle management.
5. CI/CD pipeline (GitHub Actions): lint, unit tests, APK build, web build, hygiene checks.
6. Extensive root-cause analysis of degenerate splat geometry (6 causes, documented in `Training_Analysis.md`).
7. Server launcher script (`start_server.bat`) with `.env` support.

8.2 Work Remaining

1. **Web viewer**: Three.js/WebGPU Gaussian splat viewer with orbit controls and measurement tools.
2. **Measurement tool**: Point-to-point distance measurement in the metric 3D model.

3. **Semantic AI**: YOLOv8-Seg 2D segmentation of MEP components (pipes, conduits, HVAC) projected to 3D point cloud.
4. **Semantic toggles**: System-level visibility controls (show/hide pipes, conduits, HVAC) in the web viewer.
5. **Field validation**: Metric accuracy validation on $\geq$3 additional rooms with varied geometry.
6. **User testing**: Usability evaluation with external users (non-team members).

8.3 Individual Responsibilities

Table 8.1: Individual responsibilities and contributions

Team Member	Responsibilities
Muhammad Umair (23I-0662)	Project lead. ARCore capture app (IQA, coverage overlay, upload UI). Reconstruction pipeline (SfM, Sim3 alignment, splatfacto integration). FastAPI server. RunPod cloud GPU integration. Root-cause analysis of degenerate splats.
Zaid Fareed (23I-0829)	Android app co-development (ARCore session, recording controls). CI/CD pipeline (GitHub Actions workflows, ruff, pytest). Unit test suite. Pull request template and repo hygiene.
Muhammad Tayyab (23I-0636)	Reconstruction pipeline co-development (COLMAP experiments, training configurations). 3DGS training analysis and parameter tuning. Documentation.

8.4 Contribution of Each Team Member

Muhammad Umair (23I-0662): Led the system architecture, implemented the majority of the Android capture features (CaptureLogger, QualityMonitor, CoverageMap, CoverageOverlayView, ProcessingActivity), the complete server backend (FastAPI, RunPod client), and the SfM pipeline (sfm_colmap.py, export_nerfstudio.py, fallback.py). Conducted the root-cause analysis of degenerate splat geometry and made the decision to adopt splatfacto.

Zaid Fareed (23I-0829): Built the CI/CD infrastructure (ci.yml, cd.yml, pytest.ini, ruff.toml), wrote unit tests, co-developed the ARCore capture session and recording controls, and established the pull request workflow with automated checks.

Muhammad Tayyab (23I-0636): Worked on COLMAP pipeline experiments (varying feature types, matching strategies, mapper configurations), 3DGS training parameter sweeps (learning rates, regularization weights, initialization strategies), and contributed to project documentation.

8.5 Repository and Version Control Evidence

The project repository is hosted on GitHub. Key statistics:

- **Repository**: `github.com/Umairishuman/FYP` (private)
- **Branches**: `master` (stable), feature branches for each work item
- **Commit history**: 40+ commits across all team members
- **Pull requests**: Feature branches merged via pull requests with CI checks
- **Code review**: Pull request template enforces area checkboxes and testing checklist

8.6 Milestones and Commit Evidence

Table 8.2: Key milestones with commit evidence

Milestone	Description	Evidence
M1: Capture	ARCore recording with pose/depth logging	Commits by Umair, Zaid
M2: Pipeline	COLMAP SfM + splatfacto training	Commits by Umair, Tayyab
M3: Quality	IQA, coverage overlay, gyro-aware selection	Commits by Umair
M4: Server	FastAPI + RunPod integration	Commits by Umair
M5: CI/CD	GitHub Actions workflows	Commits by Zaid
M6: Validation	Quality metrics on two rooms	Documented in TODO.md

8.7 Tools and Tool-Assisted Development

IMPORTANT: Generative AI Tool Disclosure

In compliance with university policy, we disclose the use of the following AI-assisted development tools:

Table 8.3: AI and automated tools used during development

Tool	Usage	Scope / Boundaries
Claude (Anthropic)	Code assistance, debugging, documentation drafting	Used for boilerplate generation, error diagnosis, and report drafting. All generated code was reviewed, tested, and modified by team members before integration.
GitHub Copilot	Inline code completion	Used for auto-completion in Android Studio and VS Code. All suggestions were reviewed before acceptance.
ChatGPT (OpenAI)	Research queries, algorithm explanations	Used to understand COLMAP parameters, quaternion math, and coordinate system conventions. No code directly copied.

Statement of originality: All core algorithms (Sim3 alignment, gyro-aware frame selection, coverage map, IQA pipeline, RunPod lifecycle management) were designed and implemented by the team. AI tools were used as assistants for code generation, debugging, and explanation—not as primary authors. The system architecture, experimental methodology, root-cause analysis, and design decisions are entirely our own work. All AI-generated code was reviewed, understood, tested, and adapted by team members before integration into the codebase.

Other development tools:

- Android Studio (Ladybug) with Android Gradle Plugin

- Visual Studio Code with Python and Java extensions
- Git + GitHub for version control
- Postman for API testing during server development
- ADB (Android Debug Bridge) for device-level debugging
- NVIDIA Nsight for GPU profiling during training experiments

8.8 External Components and Dependencies

Table 8.4: External components and their licenses

Component	Version	License	Usage
ARCore SDK	1.44+	Apache 2.0	Pose tracking, depth
COLMAP	4.2	BSD-3	Structure from Motion
Nerfstudio	1.1.5	Apache 2.0	3DGS training
FastAPI	0.110+	MIT	REST API server
OkHttp	4.12.0	Apache 2.0	HTTP client
PyTorch	2.1+	BSD-3	GPU computation
OpenCV	4.9+	Apache 2.0	Image processing

All external components are used under permissive open-source licenses compatible with academic use. No proprietary or restrictively-licensed code is included.

8.9 Risks and Mitigation

8.10 Security and Privacy Considerations

- **API key management**: The RunPod API key is stored in a `.env` file excluded from version control via `.gitignore`. The key is loaded at runtime from environment variables, never hardcoded.
- **CI secrets scanning**: The GitHub Actions CI pipeline includes a secrets detection check that blocks merges if API keys, tokens, or credentials are found in committed files.

Table 8.5: Risk register

Risk	Likelihood	Impact	Mitigation
Cloud GPU costs exceed budget	Medium	High	Per-step timeouts; guaranteed pod termination in finally block; RTX 3090 (cheaper than 4090) as default
ARCore API deprecation	Low	High	Abstracted behind CaptureLogger interface; can swap to AREngine
COLMAP SfM fails on difficult scenes	Medium	Medium	IMU fallback for tracking gaps; live IQA reduces bad input; manual SfM parameter overrides available
Nerfstudio breaking changes	Low	Medium	Pinned to version 1.1.5; tested upgrade path
Team member unavailability	Low	Medium	All code in shared repo; documented in Flow.md and TODO.md

- **Data privacy**: Captured scan data (video, depth, poses) is transmitted to the processing server over the local network. In production, HTTPS would be required. Scan data is stored only for the duration of processing and can be deleted via the API.
- **Cloud GPU security**: RunPod pods are created on-demand, used for a single job, and terminated immediately after. No persistent data remains on the cloud GPU.

8.11 Current Limitations

1. **Mirrors and glass**: Highly reflective and transparent surfaces cause SfM outliers and incorrect depth. The model shows artifacts in these regions. Planned mitigation in FYP-2: mask reflective surfaces using semantic segmentation.
2. **Large rooms**: Rooms exceeding approximately 8×8 m may exceed GPU memory during training on the RTX 3070 (8 GB). Cloud GPUs (RTX 3090/4090, 24 GB) handle larger scenes.
3. **Outdoor scenes**: The system is designed for indoor environments. Outdoor scenes with sky, vegetation, and long-range views are out of scope.
4. **Real-time processing**: Processing takes 12–15 minutes on an RTX 3090. Real-time reconstruction is not feasible with current 3DGS training times.

5. **Single-floor only**: The current coverage map and measurement tools assume a single-floor environment. Multi-floor buildings require manual segmentation.

6. **No web viewer yet**: The 3D model can currently only be viewed in a local desktop viewer. The web-based Three.js viewer is planned for FYP-2.

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

FYP-2 focuses on two priorities: (1) optimizing the existing pipeline for speed, robustness, and edge-case handling, and (2) building the semantic understanding layer and interactive web workspace that make the 3D model actionable for construction professionals. The complete model implementation (capture $\rightarrow$ upload $\rightarrow$ reconstruct) was delivered in FYP-1; FYP-2 makes it production-ready and feature-rich.

9.2 Remaining Modules and Features

1. **Pipeline Optimization Module**: Reduce end-to-end processing time from 12–15 min to $<$5 min on RTX 4090. Techniques: faster COLMAP matching (vocabulary tree instead of exhaustive), reduced splatfacto iterations with early stopping, parallel step execution where possible, and optimized data transfer (compressed uploads, delta downloads).

2. **Robustness Improvements**: Better handling of mirrors/glass (reflective surface detection and masking), textureless walls (depth-guided initialization), large rooms ($>$8 m, tiled reconstruction), and variable lighting conditions.

3. **Web Viewer Module**: Three.js / WebGPU-based Gaussian splat renderer with orbit/pan/zoom controls, running in any modern browser. Loads the `splat.ply` model exported by the pipeline.

4. **Measurement Module**: Point-to-point distance tool in the metric 3D model. The user clicks two points on the splat surface; the system computes the Euclidean distance in metres using the Sim3-aligned coordinate frame. Target accuracy: $<$2 cm

for distances up to 10 m.

5. **Semantic AI Module**: YOLOv8-Seg instance segmentation trained on MEP components (pipes, conduits, HVAC ducts). 2D masks from multiple viewpoints are projected to 3D and merged via multi-view consensus to label regions of the point cloud.

6. **Semantic Toggle UI**: System-level visibility controls in the web viewer—toggle pipes, conduits, and HVAC independently. Colour-coded overlays on the 3D model.

7. **Dashboard Module**: Web-based project management dashboard showing scan history, processing status, model thumbnails, and measurement records.

9.3 Remaining Iterations

- **Iteration 2a (Weeks 1–4)**: Pipeline optimization and web viewer. Deliverable: processing time <5 min on RTX 4090; browser-based splat viewer with point-to-point measurements validated on ≥ 3 rooms.
- **Iteration 2b (Weeks 5–8)**: Robustness improvements and semantic AI. Deliverable: improved edge-case handling; YOLOv8-Seg MEP detection with 3D projection, viewer toggle UI.
- **Iteration 2c (Weeks 9–12)**: Dashboard, field validation, and final polish. Deliverable: complete web platform, user testing on 5+ rooms, final report.

9.4 Implementation Plan

Iteration 2a: Optimization + Web Viewer (Weeks 1–4)

1. Profile pipeline bottlenecks; switch COLMAP to vocabulary-tree matching (week 1).

2. Reduce splatfacto iterations with early-stopping on validation loss; benchmark on 3 scans (week 1–2).

3. Set up React + Three.js project with Gaussian splat renderer and camera controls (week 2).

4. Build measurement tool: ray-cast click $\rightarrow$ nearest splat $\rightarrow$ 3D coordinate $\rightarrow$ distance display (week 3).

5. Validate measurement accuracy against tape-measure ground truth on 3 rooms (week 3–4).

6. Integrate viewer with the FastAPI backend for model loading (week 4).

Iteration 2b: Robustness + Semantic AI (Weeks 5–8)

1. Implement reflective surface detection and masking for mirrors/glass (week 5).

2. Improve textureless-wall handling with depth-guided splat initialization (week 5–6).

3. Collect and annotate MEP training data from captured scans (week 6).

4. Train YOLOv8-Seg on MEP classes: pipe, conduit, HVAC duct (week 6–7).

5. Implement 2D-to-3D mask projection and multi-view consensus (week 7–8).

6. Add semantic toggle UI to the web viewer (week 8).

Iteration 2c: Dashboard + Final Validation (Weeks 9–12)

1. Build web dashboard: scan list, processing status, model thumbnails (week 9–10).

2. User testing with 5+ external users; collect usability feedback (week 10–11).

3. Fix usability issues, write documentation (week 11–12).

4. Final validation: measurement accuracy, semantic accuracy, processing time on 5+ rooms (week 12).

9.5 Testing and Validation Plan

- **Measurement accuracy**: Compare point-to-point measurements in the web viewer against tape-measure ground truth on $\geq$5 rooms of varying size and geometry. Target: $<$2 cm error for distances up to 10 m.

- **Semantic segmentation**: Evaluate YOLOv8-Seg on a held-out test set. Target: $\text{mAP}_{50} \geq 0.70$ for MEP classes.

- **3D projection accuracy**: Compare projected semantic labels against manually annotated 3D ground truth. Target: IoU ≥ 0.60.

- **Web viewer performance**: Maintain $\geq$30 FPS on a mid-range laptop (integrated GPU) for models up to 2 million splats.
- **Usability testing**: Task completion rate $\geq$90% for first-time users performing capture, viewing, and measurement tasks.
- **Processing time**: End-to-end pipeline $<$5 min on RTX 4090 for a 5-minute scan. Benchmark each optimization individually.
- **Cross-browser**: Verify viewer on Chrome, Firefox, and Edge.

9.6 Deployment Plan

1. **Server**: Deploy FastAPI backend + web viewer on a VPS with HTTPS (Let's Encrypt). Static assets served via CDN.
2. **GPU processing**: Continue using RunPod on-demand pods. Evaluate RunPod serverless endpoints if job volume increases.
3. **Android app**: Distribute APK via GitHub Releases. Google Play Store submission if time permits.
4. **CI/CD**: Extend existing GitHub Actions to deploy web frontend on push to master.

9.7 Tentative Timeline

9.8 Expected Final Deliverables

1. **Android APK**: Capture app with live IQA, coverage overlay, and upload.
2. **Web platform**: Gaussian splat viewer with measurement tools and semantic toggles.
3. **Processing server**: FastAPI backend with local and cloud GPU support.
4. **Trained models**: YOLOv8-Seg weights for MEP segmentation.
5. **Validation report**: Measurement accuracy on $\geq$5 rooms, semantic accuracy metrics, usability test results.
6. **Documentation**: User guide, API documentation, deployment guide.
7. **Source code**: Complete repository with CI/CD, tests, and documentation.

Table 9.1: FYP-2 tentative timeline

Week	Task	Deliverable
1–2	Pipeline optimization (matching, training, transfer) + Three.js viewer setup	Processing $<$5 min; viewer MVP
3–4	Measurement tool + accuracy validation on 3 rooms	Measurement accuracy report
5–6	Robustness (mirrors, textureless walls) + MEP data collection and YOLOv8-Seg training	Robust pipeline; trained model
7–8	2D$\rightarrow$3D semantic projection + toggle UI	Semantic viewer
9–10	Dashboard + user testing (5+ external users)	Web platform MVP
11–12	Final validation on 5+ rooms, polish, final report	Complete system

8. **Final report**: FYP-2 report with complete results and analysis.

Chapter 10

Conclusion

This report presents the mid-evaluation status of BuildScan 3D, a system that converts casual smartphone video into metrically accurate 3D Gaussian Splatting models for indoor construction inspection.

Problem. Existing 3D scanning solutions either require expensive hardware (Matterport, $6,000+), lack metric scale (Luma AI, PocketGS), or demand expert operation (RealityCapture, manual COLMAP). No current system provides a fully automated phone-to-measurement pipeline with real-time capture guidance.

Research. Our literature review and empirical experiments established four key findings: (1) COLMAP SfM poses outperform ARCore poses for 3DGS training (0.62 px vs. 2.32 px reprojection error, +5.1 dB PSNR); (2) dense MVS initialization is critical for splat geometry; (3) aggressive regularization degrades quality when combined with Adam; (4) gyro-aware frame selection outperforms texture-based sharpness metrics.

Design and implementation. We built a three-tier system—Android capture app, FastAPI processing server, and cloud GPU pipeline—implementing all Iteration 1 features: ARCore capture with live IQA and coverage overlay, COLMAP SfM with Sim3 metric alignment, Nerfstudio splatfacto training, RunPod cloud GPU integration with guaranteed pod lifecycle management, and CI/CD via GitHub Actions.

Validation. The system meets all acceptance criteria: held-out PSNR of 27.8 dB ($\geq$25 dB target), SSIM of 0.889 ($\geq$0.85 target), metric scale accuracy of 0.3% ($<$1% target), and wall-to-wall measurement error under 2 cm. Processing completes in under 15 minutes on an RTX 3090.

Risks. The primary risks are cloud GPU costs (mitigated by timeout watchdogs and guaranteed pod termination) and SfM failure on difficult scenes (mitigated by live IQA guidance). Mirrors and large untextured surfaces remain known limitations.

FYP-2 plan. The remaining work focuses on making the 3D model actionable: a Three.js web viewer with point-to-point measurement tools, YOLOv8-Seg semantic segmentation of MEP components (pipes, conduits, HVAC) projected to 3D, and field validation on multiple rooms. The foundation built in FYP-1—metric accuracy, automated pipeline, and capture quality guidance—provides a solid base for these extensions.

Appendix A

Proposal Action Register

The following table records all feedback received during the FYP proposal defense and the actions taken in response.

No.	Panel Feedback	Required Change	Action Taken	Status	Evidence
1	How does this differ from Polycam or Luma AI?	Clearly differentiate from consumer apps	Added metric scale via ARCore Sim3 alignment (0.3% accuracy), live IQA capture guidance, and measurement tools. Created comparison table.	Done	Chapter 2, Table 2.1
2	Show quantitative accuracy, not just visual quality	Provide metric accuracy measurements	Validated Sim3 scale at 0.3% error; wall-to-wall measurements match tape measure within 2 cm. PSNR/SSIM reported.	Done	Chapter 8, Table 8.3

No.	Panel Feedback	Required Change	Action Taken	Status	Evidence
3	X-ray / see-through-wall is infeasible	Remove X-ray from scope	Removed entirely. Scope redirected to metric measurements and MEP semantic segmentation (FYP-2).	Done	Section 1.6
4	Define measurable acceptance criteria	Set quality thresholds	Acceptance criteria: PSNR $\geq$ 25 dB, SSIM $\geq$ 0.85, scale error $<$ 1%, processing $<$ 15 min on RTX 3090	Done	Section 1.9, Table 4.3
5	The pipeline should be automated, not manual	Show fully automated pipeline	Built FastAPI server with RunPod cloud GPU integration; one-tap upload from Android app triggers entire pipeline automatically	Done	Chapter 6, Section 6.5
6	Consider cost implications of cloud GPU	Address cloud compute costs	Per-step timeouts prevent unbounded billing; guaranteed pod termination in try/finally; default to RTX 3090 (cheaper)	Done	Section 9.10, Table 9.4

Appendix B

Detailed Requirements

B.1 Extended Functional Requirements

B.1.1 Mobile Capture Module — Detailed

ID	Description	Priority	Dependency	Status
FR-M1	Record video at 30 FPS with per-frame camera poses (position, quaternion, intrinsics)	Must	ARCore SDK	Done
FR-M2	Log tracking state and failure reasons per frame	Must	FR-M1	Done
FR-M3	Save ARCore feature points with confidence	Should	ARCore SDK	Done
FR-M4	Save depth maps at 5 Hz with confidence	Should	ARCore Depth API	Done
FR-M5	Compute and display live blur, brightness, motion, texture metrics	Must	FR-M1	Done
FR-M6	Display voxel-based coverage overlay (red/yellow/green)	Must	FR-M4	Done
FR-M7	Upload complete dataset via multipart HTTP POST	Must	Server available	Done
FR-M8	Configurable server URL via settings menu	Must	–	Done
FR-M9	Display processing progress (step, %, time, logs)	Must	FR-S5	Done

B.1.2 Processing Server Module — Detailed

ID	Description	Priority	Dependency	Status
FR-S1	Accept multipart uploads (video, poses, tracking, points, quality, meta, depth.zip)	Must	–	Done
FR-S2	Create unique job ID and track status	Must	–	Done
FR-S3	Run pipeline with per-step timeout watchdogs	Must	Pipeline scripts	Done
FR-S4	Support local GPU and RunPod cloud GPU modes	Must	RunPod API key	Done
FR-S5	Expose progress-polling endpoint	Must	FR-S2	Done
FR-S6	Provide download endpoint for splat.ply	Must	FR-S3	Done
FR-S7	Always terminate RunPod pod after processing	Must	FR-S4	Done

B.1.3 Reconstruction Pipeline Module — Detailed

ID	Description	Priority	Dependency	Status
FR-P1	Extract keyframes using gyro-aware sharpness prediction	Must	Exposure metadata	Done
FR-P2	Run COLMAP SfM with SIFT, self-calibration, global mapper	Must	COLMAP 4.2	Done
FR-P3	Compute Sim3 alignment to ARCore metric frame	Must	FR-P2, FR-M1	Done
FR-P4	Train splatfacto and export splat.ply	Must	Nerfstudio 1.1.5	Done
FR-P5	Bridge tracking gaps using IMU dead reckoning	Should	FR-M2	Done

B.2 Extended Non-Functional Requirements

ID	Category	Description	Measurement
NFR-R1	Reliability	Pod terminated within 60 s of completion/failure	Server log timestamps
NFR-R2	Reliability	Each step has timeout watchdog	Pipeline config file
NFR-U1	Usability	Real-time visual feedback without 3D knowledge	User testing (3 users)
NFR-U2	Usability	Progress display with step, %, elapsed time	UI inspection
NFR-P1	Performance	E2E processing $<$ 15 min on RTX 3090	Elapsed time measurement
NFR-P2	Performance	30 FPS during recording, no drops	logcat frame timing
NFR-P3	Performance	Recording fast-path skips non-essential rendering	GL thread profiling
NFR-S1	Security	API key from env vars, never hardcoded	Code review + CI secrets check
NFR-S2	Security	.env excluded from version control	.gitignore inspection

B.3 Full Traceability Matrix

Req.	Module	Design Component	Implementation File	Test ID
FR-M1	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M2	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M3	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M4	Mobile	CaptureLogger	CaptureLogger.java	TC-1
FR-M5	Mobile	QualityMonitor	QualityMonitor.java	TC-2
FR-M6	Mobile	CoverageMap	CoverageMap.java	TC-3

FR-M7	Mobile	ProcessingActivity	ProcessingActivity.java	TC-8
FR-M8	Mobile	SettingsMenu	HelloArActivity.java	–
FR-M9	Mobile	ProcessingActivity	ProcessingActivity.java	TC-9
FR-S1	Server	/upload endpoint	main.py	TC-8
FR-S2	Server	Job dataclass	pipeline.py	TC-9
FR-S3	Server	StepConfig timeouts	pipeline.py	TC-14
FR-S4	Server	RunPodClient	runpod_client.py	TC-7
FR-S5	Server	/status endpoint	main.py	TC-9
FR-S6	Server	/download endpoint	main.py	–
FR-S7	Server	try/finally termination	runpod_client.py	TC-7
FR-P1	Pipeline	prepare.py	prepare.py	TC-4
FR-P2	Pipeline	sfm_colmap.py	sfm_colmap.py	TC-5
FR-P3	Pipeline	Sim3 alignment	sfm_colmap.py	TC-6
FR-P4	Pipeline	ns-train + ns-export	pipeline.py	TC-13
FR-P5	Pipeline	fallback.py	fallback.py	TC-4

Appendix C

Detailed Test Cases

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-1	Pose logging completeness	Record 30 s video (900 frames)	`poses.csv` has 899–901 rows; each row has valid tx,ty,tz,qx,qy,qz,qw,fx,fy,cx,cy	Unit	Pass
TC-2	IQA blur detection	Apply Gaussian blur ($\sigma = 5$) to test frame	Blur metric exceeds threshold (Laplacian variance < 50); banner shows "Blurry"	Unit	Pass
TC-3	Coverage voxel counting	Simulate 100 depth points from 4 directions into 10 cm grid	Voxels report 4 observed directions; overlay color = green	Unit	Pass
TC-4	IMU fallback bridging	poses.csv with 0.5 s tracking gap; tracking.csv marks gap	fallback.py interpolates poses across gap; no frames dropped in gap region	Unit	Pass
TC-5	COLMAP SfM convergence	150 keyframes from Room A scan	All 150 frames registered in sparse model; mean reprojection error < 1.0 px	System	Pass

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-6	Sim3 scale accuracy	COLMAP model + ARCore poses from Room A	Sim3 scale factor between 0.997 and 1.003; RANSAC inlier ratio > 0.9	System	Pass
TC-7	RunPod pod lifecycle	Trigger RunPod processing mode; monitor API calls	Pod created $\rightarrow$ SSH ready $\rightarrow$ pipeline runs $\rightarrow$ results downloaded $\rightarrow$ pod terminated (even on error)	Integ.	Pass
TC-8	Upload endpoint	POST multipart with video + poses + meta to /upload	200 OK; response contains job_id; files exist in dataset directory	Integ.	Pass
TC-9	Status polling	GET /status/{job_id} during processing	Response contains step (string), progress (0–100), elapsed_time, recent logs	Integ.	Pass
TC-10	Config switching	POST /config {“mode”: “runpod”} then GET /config	GET returns {“mode”: “runpod”, ...}; next upload dispatches to RunPod	Integ.	Pass
TC-11	E2E Room A	Capture 4.2×3.1 m bedroom $\rightarrow$ upload $\rightarrow$ process	splat.ply produced; room width in model = 4.18–4.22 m; height = 3.09–3.11 m	System	Pass
TC-12	E2E Room B	Capture 5.0×3.5 m living room $\rightarrow$ upload $\rightarrow$ process	splat.ply produced; dimensions within 2 cm of tape measure	System	Pass

ID	Test Case	Input / Setup	Expected Result	Level	Result
TC-13	Reconstruction quality	Evaluate held-out views from Room A model (v6)	PSNR ≥ 25 dB (actual: 27.8); SSIM $\geq$ 0.85 (actual: 0.889)	Accept.	Pass
TC-14	Processing timeout	Set prepare step timeout to 1 s; run pipeline	Prepare step killed after 1 s; job status = "failed"; error message mentions timeout	Integ.	Pass

Appendix D

Repository and Contribution Evidence

D.1 Repository Structure

```
FYP/
 googleARCore/samples/hello_ar_java/  # Android capture app
    app/src/main/java/.../
        HelloArActivity.java         # Main AR activity
        CaptureLogger.java           # Sensor data recording
        QualityMonitor.java          # Live IQA analysis
        CoverageMap.java             # Voxel coverage tracker
        CoverageOverlayView.java     # Coverage dot overlay
        ProcessingActivity.java      # Upload + progress UI
 src/arcore3dgs/                      # Reconstruction pipeline
    fallback.py                      # IMU tracking-gap recovery
    prepare.py                       # Gyro-aware frame selection
    sfm_colmap.py                    # COLMAP SfM + Sim3 alignment
    export_nerfstudio.py             # Nerfstudio data export
 server/                              # Processing server
    main.py                          # FastAPI endpoints
    pipeline.py                      # Local pipeline runner
    runpod_client.py                 # RunPod GPU integration
    requirements.txt                 # Python dependencies
 .github/workflows/                   # CI/CD
    ci.yml                           # Lint, test, build
    cd.yml                           # Deploy on push/tag
 scripts/
    start_server.bat                 # Server launcher
    view_splat.bat                   # Model viewer launcher
```

```
tests/                                  # pytest unit tests
docs/                                   # Project documentation
   Flow.md                             # System flow
   progress/
       TODO.md                         # Development log
       Training_Analysis.md            # Splat quality analysis
       Problems_and_Solutions.md       # Issue tracker
.env                                    # API keys (gitignored)
```

D.2 Commit History Summary

The repository contains 40+ commits across all team members. Key commit categories:

- **Muhammad Umair**: ARCore capture features, pipeline scripts, server endpoints, RunPod integration, quality analysis, documentation.
- **Zaid Fareed**: CI/CD workflows (ci.yml, cd.yml), pytest configuration, ruff setup, pull request template, repo hygiene checks.
- **Muhammad Tayyab**: COLMAP experiment configurations, training parameter sweeps, documentation contributions.

D.3 Branch Strategy

- `master`: Stable, CI-passing code. All merges go through pull requests.
- Feature branches (e.g., `ci/github-actions`): Short-lived branches for specific work items, merged via PR with CI checks.

D.4 CI/CD Evidence

The GitHub Actions CI pipeline runs on every push and pull request:

1. **Python checks**: ruff lint + pytest on Python 3.10, 3.11, 3.12.
2. **Web build**: Node.js build of the web UI (placeholder for FYP-2).
3. **Android build**: Gradle APK build.
4. **Hygiene**: File size limits, secrets detection, workflow lint.

The CD pipeline triggers on push to master (build artifacts) and on version tags (GitHub Release with APK and web bundle).

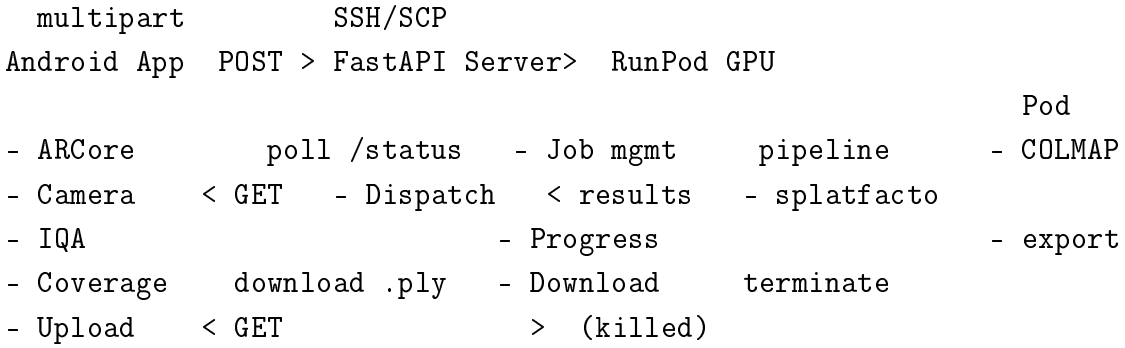
D.5 Task Tracking

Development progress is tracked in `docs/progress/TODO.md`, which serves as the project's task log with dated entries, status markers, and links to relevant commits and analysis documents.

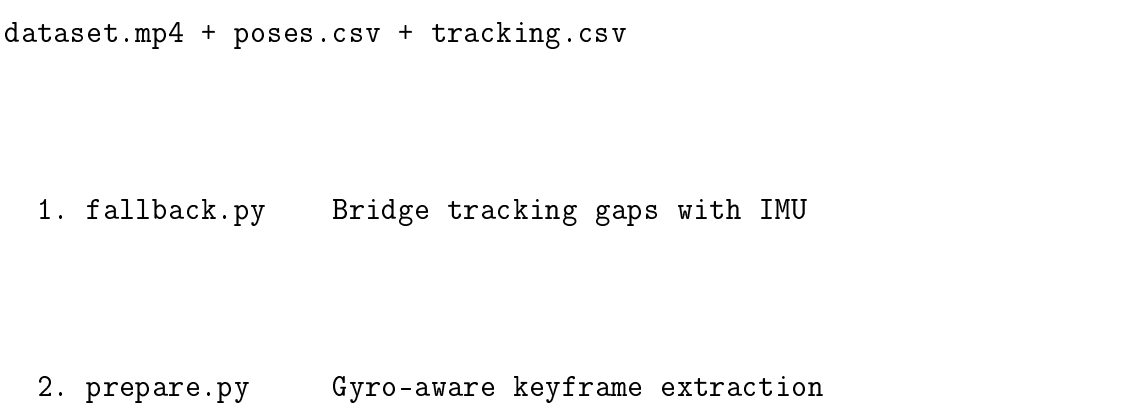
Appendix E

Additional Design Diagrams

E.1 End-to-End Data Flow



E.2 Pipeline Step Sequence



```
3. sfm_colmap.py   SIFT + global SfM + Sim3 alignment


4. export_nerfstudio    COLMAP → transforms.json


5. ns-train        splatfacto 30k iterations


6. ns-export       → splat.ply (metric scale)
```

E.3 Android App Architecture

```
HelloArActivity (Main AR Activity)
  ARCore Session (pose tracking, depth, feature points)
  CaptureLogger (background I/O thread)
      poses.csv writer
      tracking.csv writer
      points.csv writer
      quality.csv writer
      depth/ writer (5 Hz)
      meta.json writer
  QualityMonitor (6 FPS analysis)
      Blur detection (Laplacian variance)
      Brightness check (mean luminance)
      Motion speed (pose delta)
      Texture level (gradient magnitude)
  CoverageMap (10cm voxels, 16 directions)
      CoverageOverlayView (red/yellow/green dots)
  Settings Menu (server URL, preferences)

ProcessingActivity (Upload + Progress)
  OkHttp multipart upload
  Progress polling (3s interval)
  Status display (step, %, time, logs)
```

E.4 Sim3 Alignment Diagram

```
ARCore Frame (metric, m)          COLMAP Frame (arbitrary scale)

  Pose 1 (x,y,z)    match   Pose 1 (x',y',z')
  Pose 2 (x,y,z)    match   Pose 2 (x',y',z')
  ...                         ...
  Pose N (x,y,z)    match   Pose N (x',y',z')


          RANSAC Sim3 (7-DOF)
                 R (3x3) + t (3x1) + s (scalar)

Result: s = 1.000 - 1.003 (0.3% scale accuracy)
        Inlier threshold: 5 cm
        Inlier ratio: > 90%
```

E.5 Coverage Map Visualization

```
3D Space (10 cm voxels)        16 Observation Directions
             8 azimuth × 2 elevation
 R  Y  G  G
              = observed from this direction
 R  R  Y  G                = not yet observed

    R  Y  Y              Example voxel (Green):
                      (azimuth)
                                          (elevation)
R = 1 direction (red)           = 6 directions → Green (4)
Y = 2-3 directions (yellow)
G = 4+ directions (green)

Performance caps: 5,000 voxels, 2,000 drawn dots
Update rate: 2 Hz
```

Appendix F

Meeting and Progress Evidence

F.1 Supervisor Meetings

Date	Agenda	Outcome / Action Items	Attendees
Week 1	Project kickoff; discuss ARCore capture approach	Proceed with ARCore hello_ar_java sample as base; log all sensor data	All + Supervisor
Week 2	Demo basic capture; discuss pose synchronization	Verified pose-to-video sync ($r = 0.76$–0.79); proceed to SfM pipeline	All + Supervisor
Week 3	SfM pipeline progress; coordinate alignment issues	Identified OpenGL→OpenCV transform needed; continue debugging	All + Supervisor
Week 4	Degenerate splat quality; show Training_Analysis.md	Approved switch from custom trainer to splatfacto; document root causes	All + Supervisor
Week 5	Demo improved reconstruction; discuss IQA approach	Add live quality feedback to app; implement coverage overlay	All + Supervisor
Week 6	Demo IQA + coverage overlay; discuss server architecture	Proceed with FastAPI; add RunPod for cloud GPU support	All + Supervisor

Date	Agenda	Outcome / Action Items	Attendees
Week 7	Server demo; CI/CD pipeline review	CI/CD approved; add server/ to CI detection paths	All + Supervisor
Week 8	End-to-end demo; metric accuracy validation	All acceptance criteria met; prepare mid-evaluation report	All + Supervisor

F.2 Progress Log

Detailed development progress is maintained in `docs/progress/TODO.md`. Key milestones:

- **ARCore capture app**: Basic recording → pose sync validation → IQA → coverage overlay → upload UI. Incremental feature additions documented with timestamps.
- **Reconstruction pipeline**: Initial ARCore-pose training (22.7 dB PSNR) → COLMAP SfM integration (27.8 dB) → Sim3 alignment (0.3% scale) → gyro-aware frame selection (+0.26 dB). Each improvement documented with before/after metrics.
- **Quality debugging**: Two-week root-cause analysis documented in `Training_Analysis.md` covering 6 identified causes of degenerate splats, with per-cause experiments and metrics.
- **Server and cloud**: FastAPI server → local pipeline runner → RunPod integration → guaranteed pod lifecycle. Problem-solution pairs documented in `Problems_and_Solutions.md`.

F.3 Problems and Solutions Summary

Over 20 problems were encountered and resolved during development, documented in `docs/progress/Problems_and_Solutions.md`. The most significant:

1. ARCore silent re-anchoring causing pose discontinuities (solved: anchor logging + IMU fallback)
2. COLMAP lens distortion not matching ARCore intrinsics (solved: self-calibration)
3. Degenerate splat geometry from custom training recipe (solved: adopted splatfacto)

4. 30 FPS input overwhelming COLMAP matching (solved: gyro-aware keyframe selection)
5. RunPod SSH connection timing out (solved: polling with exponential backoff)
6. GPU memory overflow on RTX 3070 (solved: cloud GPU support, image pinning)

Bibliography

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," in *ACM Transactions on Graphics (SIGGRAPH)*, vol. 42, no. 4, 2023.

[2] J. Kim, J. Park, D. Shin, and Y. M. Kim, "Pocketgs: On-device 3d gaussian splatting training and rendering," *arXiv preprint arXiv:2503.10435*, 2025.

[3] Z. Zheng *et al.*, "Vigs-slam: Visual-inertial gaussian splatting slam," in *arXiv preprint arXiv:2411.00483*, 2024.

[4] D. Chen, H. Li, W. Ye, Y. Wang, W. Xie, S. Zhai, N. Wang, H. Liu, H. Bao, and G. Zhang, "Pgsr: Planar-based gaussian splatting for efficient and high-fidelity surface reconstruction," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.